

Introduction

Our goal in this project was to build a complete big data pipeline and analysis workflow using the New York City Taxi and Limousine Commission (TLC) trip record data. The TLC publishes monthly trip records for several types of taxi and for-hire vehicle services. We used the four datasets that were relevant for our analysis: Yellow taxi (medallion cabs, primarily concentrated in Manhattan), Green taxi (street-hail livery vehicles serving mainly northern Manhattan and the outer boroughs), FHV (for-hire vehicles dispatched through licensed bases), and FHVHV (high-volume FHV services such as Uber and Lyft, reported separately from 2019 onwards) [1].

Most of our processing was done in **Snowflake** [2], which served as our main big data platform. The Snowflake setup uses the warehouse:

BIGDATA_MZMB_WH

and the database:

BIGDATA_TAXI_MZMB

This is where all our main results are available, separated into the `BRONZE`, `SILVER`, `GOLD`, and `EXTERNAL_DATA` schemas. The data pipeline follows a simple medallion structure: the `BRONZE` layer contains raw staged Parquet files and schema inspection outputs, the `SILVER` layer contains unified and cleaned trip tables, the `GOLD` layer contains aggregated results, enriched tables, and visualization inputs, and external datasets are stored in the `EXTERNAL_DATA` schema. You can log in to Snowflake to see the results at <https://sfedu02-fhb32629.snowflakecomputing.com>, and then go to the mentioned database (Catalog → Database Explorer).

The full project is also available on GitHub at: https://github.com/7matic/bd_project/tree/main/Project.

The repository contains all scripts, notebooks, SQL files, figures, and other report materials used for the project. Code is organized by task under `scripts/`, while figures used in the README and report are stored under `images/`.

In the following sections, we go through all the tasks and describe them individually, following the main CRISP-DM approach: first understanding, preparing, and exploring the data (earlier stages), then using that information to do proper modeling and evaluation.

T0 & T1: Data ingestion and schema unification

T0: Downloading, staging, and schema inspection

We initially used the shared TLC dataset on the Arnes HPC cluster at `/d/hpc/projects/FRI/bigdata/data/Taxi`. Since we already had the Parquet files up to January 2025, it was easier to obtain the remaining newer files. We downloaded the missing data up to February 2026 for all four datasets: Yellow taxi, Green taxi, FHV, and FHVHV. Missing files were downloaded using a Dask Bag workflow in `scripts/t0/missing_data.py` and saved on the same HPC cluster. We applied the map-reduce approach: the map step processed files independently, and the final step summarized how many files were downloaded, present, or missing.

After obtaining all the necessary files, we moved to Snowflake and created our main warehouse and database: `BIGDATA_MZMB_WH`, `BIGDATA_TAXI_MZMB`. Within the `BRONZE` schema, we created one internal stage per dataset: `YELLOW_RAW_STAGE`, `GREEN_RAW_STAGE`, `FHV_RAW_STAGE`, and `FHVHV_RAW_STAGE`. We used the upload script

scripts/t0/snowflake.py to collect files from both the original shared directory and the missing-data directory where we downloaded new files, and then uploaded them to the correct Snowflake stage using the PUT command.

For schema inspection, we saved results in four tables, one per dataset, and also combined them into `BRONZE.TO_SCHEMA_ALL` to provide an overview of all obtained column names, data types, null information, and source files. Raw stages and the created inspection tables can be seen in Figure 1.

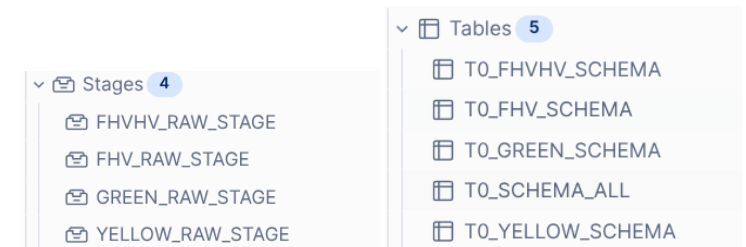


Figure 1. T0 Snowflake setup: raw stages for the four datasets and schema inspection tables in the BRONZE layer.

We quickly observed that some files are not perfectly consistent over time: some columns have changed names, some have changed types, and some new columns only started appearing in later years.

T1: Silver layer schema unification

In T1, we created the unified SILVER tables from the raw staged Parquet files. The main script is `scripts/t1/t1.sql`. Figure 2 shows an example preview of the unified Snowflake Yellow taxi table after schema differences were resolved.



Figure 2. Preview of unified records in Snowflake for Yellow taxi after schema unification.

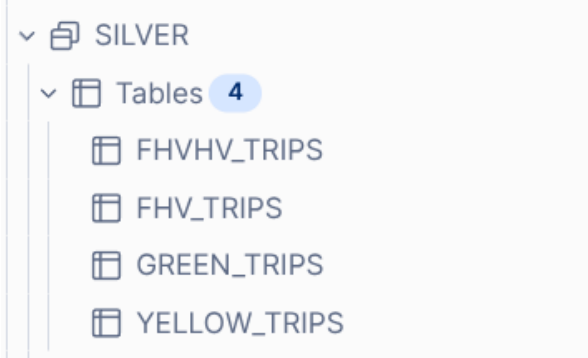
We implemented the repartitioning concept in Snowflake by clustering on pickup year and month, which achieves the same practical result as year-based repartitioning in a file-based workflow. We extracted these two columns from the pickup timestamp and used them in later tasks for filtering and aggregation as well. The main outputs are the four unified tables: `SILVER.YELLOW_TRIPS`, `SILVER.GREEN_TRIPS`, `SILVER.FHV_TRIPS`, and `SILVER.FHVVH_TRIPS`, as shown in Figure 3.

Type inconsistencies were handled with safe casts in SQL. For columns with multiple historical names, we used COALESCE. For example, the Yellow taxi data contains both `airport_fee` and `Airport_fee` depending on the year, so we merged them into one consistent column.

At this stage, we did not apply strict data quality filtering because data quality issues are measured and addressed in T2.

Table 1. Silver layer table sizes after unification.

Table	Columns	Rows
SILVER.YELLOW_TRIPS	23	~1.3B
SILVER.GREEN_TRIPS	24	~84M
SILVER.FHV_TRIPS	10	~797M
SILVER.FHVVHV_TRIPS	28	~1.52B

**Figure 3.** Unified trip tables created in the SILVER schema.

T2: Data quality analysis

Methodology

After creating the unified SILVER tables in T1, we inspected the data quality issues common in the TLC files. The main script for this task is `scripts/t2/t2.sql`. For each dataset, we created three result tables in the GOLD schema:

- `QUALITY_FILE_YEAR`: file-level quality statistics grouped by source file and pickup year,
- `QUALITY_YEAR`: yearly aggregation of the file-level results,
- `QUALITY_CHART`: table used for plotting issue percentages.

The per-file table represents the map step, and the yearly table represents the reduce step. The charts were created directly from the `QUALITY_CHART` tables in Snowflake, which is a useful feature that allows for quick visualizations directly on the platform. We used percentages in the visualizations instead of raw counts because the datasets vary greatly in size.

The exact checks regarding data quality depended on the dataset, because not all datasets have the same columns. The main checks were:

- pickup year outside the expected range for the dataset,
- pickup timestamp outside the source file month, with a one-day tolerance,
- pickup timestamp equal to dropoff timestamp,
- dropoff timestamp earlier than pickup timestamp,
- zero or non-positive distance,
- non-positive passenger count for Yellow and Green,
- negative fare or total amount,
- missing pickup/dropoff locations and missing base information for FHV/FHVVH.

Yellow and Green taxi quality

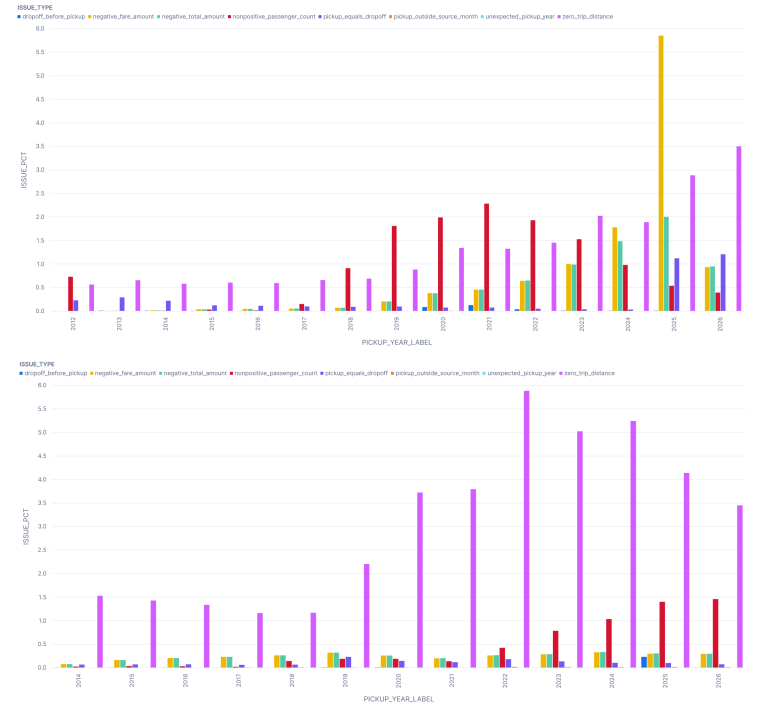
Yellow and Green taxis had similar types of issues. Most pickup years were valid, but a few unusual years appeared. For both Yellow and Green, the main problems were zero-distance trips, pickups equal to dropoffs, negative fares, and some non-positive passenger counts. Table 2 shows only some selected quality issues, as it would not be possible to display everything in a single table. Other results can be found in the README of our GitHub repository.

Figure 4 shows that the main issue percentages for Yellow and Green remain relatively small overall, but some problems increase

Table 2. Selected quality issue rates for Yellow and Green taxi.

Data	Year	Rows (M)	Zero dist.	Neg. fare	Neg. total
Yellow	2012	171.4	0.56%	0.00%	0.00%
Yellow	2020	24.6	1.34%	0.38%	0.38%
Yellow	2025	48.7	2.88%	5.85%	2.00%
Green	2014	15.8	1.52%	0.07%	0.07%
Green	2020	1.7	3.72%	0.26%	0.25%
Green	2025	0.6	4.13%	0.29%	0.30%

in later years. The increase in zero-distance most likely reflects app-canceled trips, and negative fare could be due to the growing number of digital refund entries.

**Figure 4.** Quality issue rates by year for Yellow taxi (top) and Green taxi (bottom).

Based on these results, we created stricter clean views for Yellow and Green, which are `SILVER.YELLOW_TRIPS_CLEAN` and `SILVER.GREEN_TRIPS_CLEAN`. These views keep only rows with valid pickup years, pickup timestamps within the expected file month, dropoff after pickup, positive trip distance, non-negative fare and total amount, and passenger count either null or positive.

FHV and FHVHV quality

FHV had the most quality issues. Early FHV records showed serious problems with dropoff timestamps. In 2015 and 2016, almost all records had dropoff timestamps earlier than pickup timestamps and many missing pickup and dropoff location IDs. FHVHV was much cleaner than FHV, with more complete columns and better coverage. Some issues remained, especially in 2019, such as non-positive trip miles and negative base passenger fares, but these were minor compared to the full dataset size. Some of the more noticeable issues are displayed in Table 3.

Table 3. Selected quality issue rates for FHV and FHVHV.

Data	Year	Rows (M)	Dropoff<pickup	Missing PU	Main numeric issue
FHV	2015	63.4	100.00%	27.26%	–
FHV	2016	132.1	100.00%	30.16%	–
FHV	2025	24.9	0.00%	82.02%	–
FHVHV	2019	234.6	0.00%	0.00%	0.25% neg. fare
FHVHV	2022	212.4	0.00%	0.00%	0.02% nonpos. miles
FHVHV	2025	243.6	0.01%	0.00%	0.01% nonpos. miles

Figure 5 shows the difference more clearly. For FHVHV, we created a stricter clean view because removing problematic rows did not eliminate much useful data: `SILVER.FHVHV_TRIPS_CLEAN`. This view retains valid pickup years, pickup timestamps within the expected source file month, dropoff times after pickup, positive trip miles and trip time, non-negative fare and driver pay, non-null locations, and non-empty base information. For FHV, we used a lighter clean view: `SILVER.FHV_TRIPS_CLEAN`. This view retains valid pickup years, pickup timestamps within the expected source file month, and non-empty dispatching base numbers. We did not apply strict filters to FHV, as that would remove a very large portion of the dataset.

The obtained views served as the starting point for all subsequent tasks related to exploratory analysis, augmentation, streaming, machine learning, and more.



Figure 5. Quality issue rates by year for FHV (top) and FHVHV (bottom).

T3: Exploratory data analysis

All of the exploratory data analysis was computed in Snowflake SQL using clean views generated with `scripts/t3/t3.sql`. Visualizations were produced in Snowpark Python (`scripts/t3/t3_visualizations.ipynb`).

Dataset scale

After cleaning, the total trip counts are shown in Figure 6 and Table 4.

Table 4. Total trip counts after cleaning

Service	Period	Trips (B)
FHVHV	2019–2026	1.52
Yellow	2012–2026	1.29
FHV	2015–2026	0.80
Green	2014–2026	0.08

Comparison metrics

To compare how similarly the four datasets behave, we use cosine similarity for distribution-shaped data. Cosine similarity measures

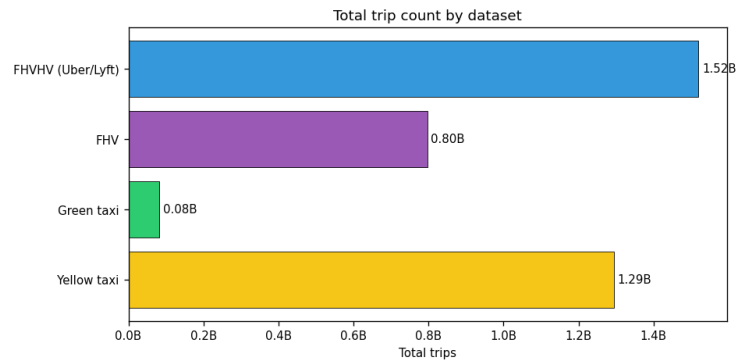


Figure 6. Bar graph of total trip counts.

the angle between two vectors rather than their magnitude, so it captures whether two distributions have the same shape regardless of how many total trips each dataset has. For monthly volume, where we care about correlated trends over time rather than shape, we use Pearson correlation instead.

Temporal analysis

Monthly volume (Figure 7) reveals three important events: (1) steady Yellow decline from 2014 as rideshare grew; (2) the February 2019 FHVHV regulation, causing FHV to collapse while FHVHV simultaneously emerged; and (3) COVID-19 (March–April 2020), causing an immediate drop in everything. Post-COVID, FHVHV reached new highs by 2022 while Yellow and Green settled at ~25% of pre-pandemic levels.

Hourly patterns (Figure 8) for all four datasets follow the same trend. Start at 05:00, morning peak at 08:00, broader evening peak at 18:00–19:00. Pairwise cosine similarity is ≥ 0.99 across all pairs.

Day-of-week patterns (Figure 9) are essentially the same for all datasets. Pairwise cosine similarities are 1.00. Friday and Saturday are busiest while Monday is quietest. Variation across days is modest (± 2 –3 percentage points).

COVID-19 impact

Monthly trips in 2020 were compared against 2019 (`GOLD.T3_COVID_IMPACT`), shown in Figure 10. In April 2020 (first full lockdown month) we see Yellow taxi fall by 96%, Green 93% and FHVHV 80%. By December 2020, Yellow and Green remained ~80% below 2019 levels while FHVHV recovered to within ~46% of 2019 volume.

Spatial analysis

Yellow pickups concentrate in Manhattan (top zones: 237 Midtown, 161 Lincoln Square, 236 Murray Hill) plus airports. Green is almost entirely outer boroughs (top: 74 East New York, 75 Flatbush, 41 Crown Heights North). FHV and FHVHV show broader citywide coverage. The top Yellow OD pair is 264→264 (~16M trips – missing location data); excluding artefacts, genuine top pairs are in the Midtown–Upper East Side corridor.

Trip characteristics

Figure 11 shows yearly trip statistics for Yellow, Green, and FHVHV. Fares rose sharply from 2022 onwards across all three datasets, due to post-pandemic demand fare adjustments. Average trip distance is stable (~2.5–3.5 miles for Yellow/Green and slightly longer for FHVHV). Trip duration increased noticeably from 2022–2023, consistent with increased traffic congestion.

Payment type is only available for the Yellow and Green dataset. Yellow is 62.9% credit card (813M trips), 35.1% cash (454M). Green is nearly split: 50.1% cash (40M), 49.4% credit card (40M), reflecting higher cash usage in outer boroughs.

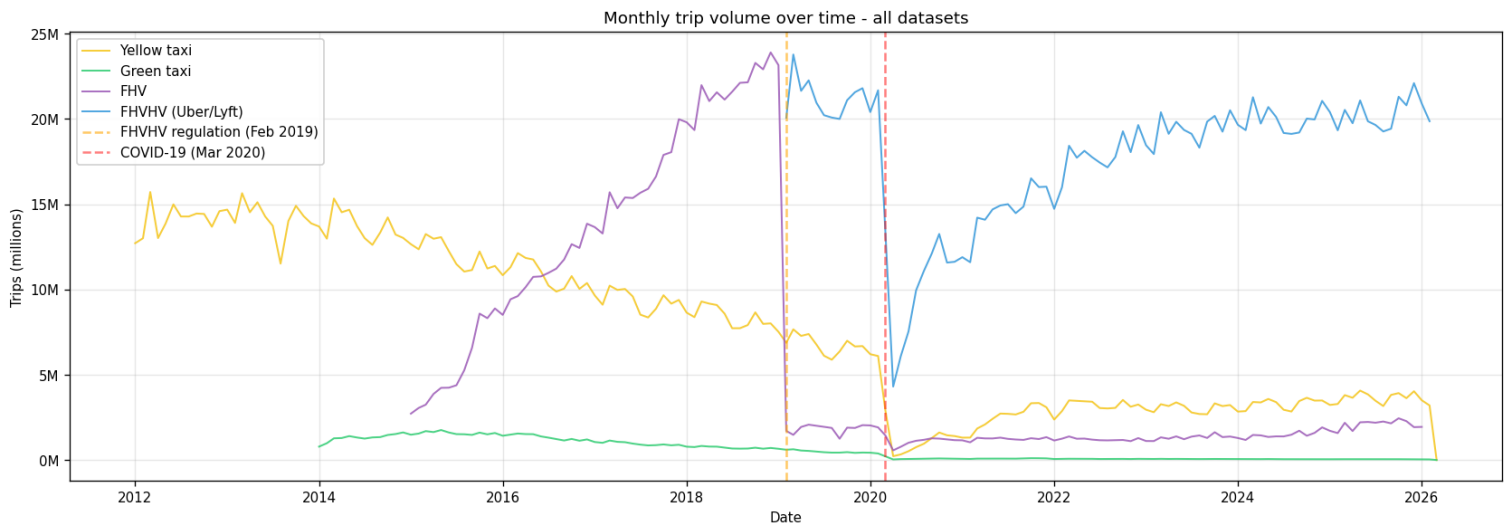


Figure 7. Monthly trip volume by dataset, 2012–2026.

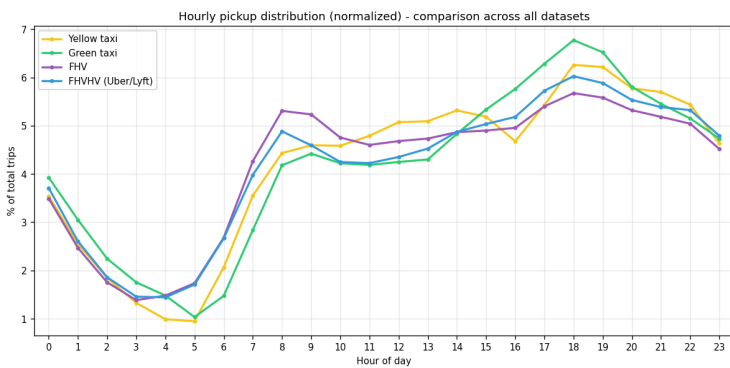


Figure 8. Normalised hourly pickup distribution.

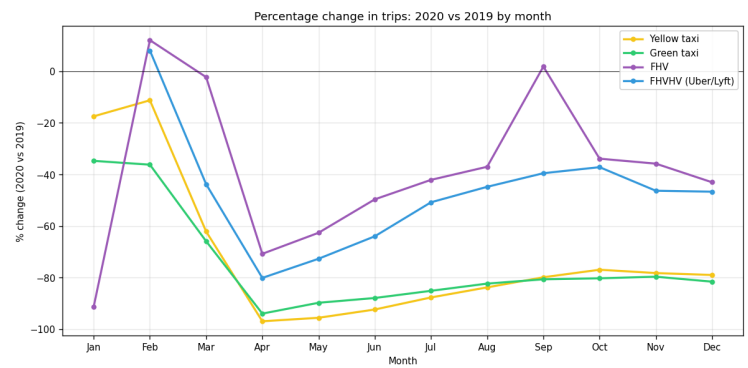


Figure 10. Monthly trip change 2020 vs. 2019 by dataset.

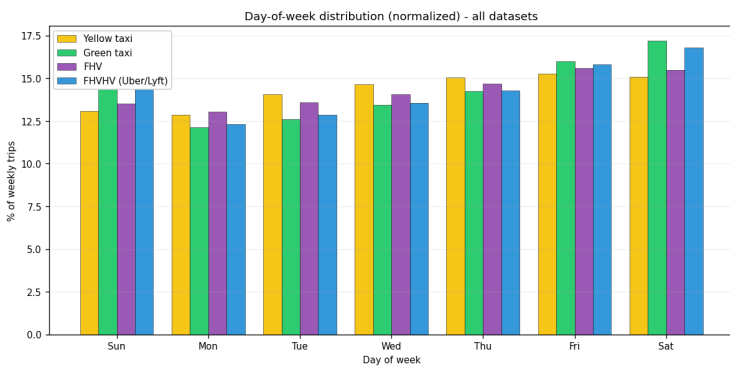


Figure 9. Normalised day-of-week distribution.

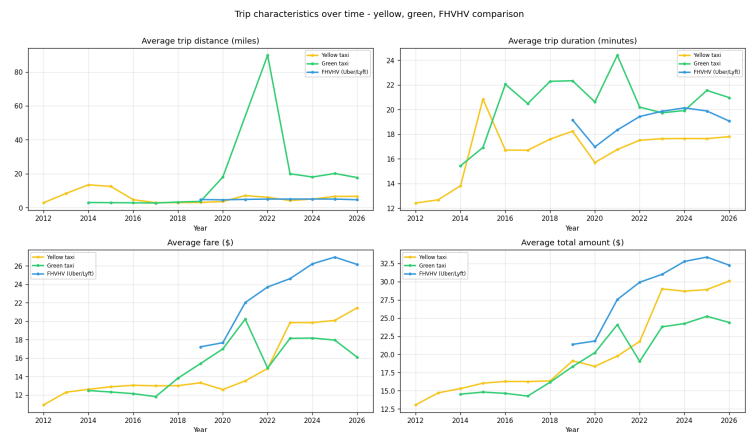


Figure 11. Trip characteristics over time: distance, duration, fare, total amount.

FHVHV: Uber vs. Lyft

Within FHVHV, Uber (HV0003) holds 70–75% of trips and Lyft (HV0005) holds 25–30% consistently across all years. Both show near-identical trends in distance, fare, and driver pay. Driver pay has grown for both since 2021, consistent with NYC minimum earnings floor regulations.

Cross-dataset similarity

Figure 12 shows pairwise similarity matrices for four metrics: cosine similarity on hourly, DOW, and spatial distributions, and Pearson correlation on monthly volume.

In terms of when trips happen, all four datasets are essentially interchangeable. Their hourly and day-of-week patterns are nearly identical, with cosine similarities at or above 0.99 in both cases. This means that regardless of which service a passenger uses, the overall

rhythm of demand across the day and week is the same.

Where the datasets differ is in where trips happen. Yellow and Green taxis have almost no geographic overlap – Yellow serves Manhattan, Green serves the outer boroughs – giving them a very low spatial similarity of 0.08. FHV and FHVHV, on the other hand, both operate citywide, so their spatial similarity is much higher at 0.85.

For monthly volume trends, we use Pearson r , which measures how closely two series rise and fall together (1.0 = perfectly in sync, 0.0 = no relationship). Yellow and Green move together most closely ($r = 0.95$): both have been declining in parallel as rideshare took over. FHVHV and Green are the least correlated ($r = 0.33$) because they have gone in opposite directions.

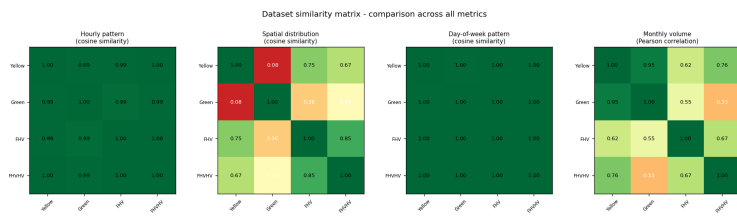


Figure 12. Cross-dataset similarity matrices: hourly (cosine), DOW (cosine), spatial (cosine), monthly volume (Pearson r).

T4: Storage format comparison

Experimental setup

We benchmarked five storage formats using the Green 2024 partition (617,885 rows, 24 columns) from `SILVER.GREEN_TRIPS_CLEAN`: Parquet with snappy compression (PyArrow [3]), plain CSV (pandas), gzip-compressed CSV (pandas), HDF5 with gzip compression (h5py), and DuckDB. Snowflake supports Parquet, CSV, and gzip CSV natively via `COPY INTO` stage exports. HDF5 and DuckDB were benchmarked locally only, as neither can be installed in Snowflake’s Snowpark environment.

Measurement methodology

For Python, write time was measured as a single run and read time as the median of three runs from disk. HDF5 used h5py with gzip compression for numeric columns and fixed-length byte strings for text and datetime fields.

For Snowflake, write time was the duration of the `COPY INTO` query and read time was the duration of a `COUNT (*)` scan on the staged files. A full `COUNT (*)` was chosen over `SELECT *` because staged CSV files carry no schema metadata. Snowflake cannot infer column names without explicit positional references, so `COUNT (*)` was used consistently across all formats to force a full parse.

Results

Table 5 and Figure 13 show the benchmark results.

Table 5. Format benchmark

Format	Size (MB)	Read (s)	Write (s)
Parquet [Python]	15.1	0.11	0.82
CSV [Python]	90.4	1.21	6.84
CSV.gz [Python]	13.2	1.40	18.48
HDF5 [Python]	38.9	0.30	0.95
DuckDB [Local]	23.6	0.62	1.76
Parquet [Snowflake]	14.4	0.42	2.12
CSV [Snowflake]	94.6	0.52	2.86
CSV.gz [Snowflake]	14.0	0.60	3.23

Parquet (snappy) is the clear winner for analytical workloads: 0.11 s read time (11× faster than plain CSV), a competitive 15.1 MB file size, and a fast 0.82 s write. Its columnar layout lets readers skip columns they do not need, and snappy compression is cheap to decompress.

Gzip CSV achieves the smallest file size (13.2 MB) but at a steep cost: write takes 18.5 s, over 22× slower than Parquet, because gzip compression is single-threaded and CPU-intensive. This format makes sense only for long-term archival where read and write speed are secondary concerns.

Plain CSV is the weakest option overall, it is 6× larger than Parquet with no speed advantage. Its only practical benefit is broad tool compatibility.

HDF5 sits in the middle: 38.9 MB, 0.30 s read, and 0.95 s write. It is a reasonable choice for Python-only workflows, but it is unsupported

in Snowflake and requires more complex code to read and write than Parquet or CSV.

DuckDB was benchmarked locally since it cannot be installed in Snowflake’s Snowpark environment. It produced a 23.6 MB file, read in 0.62 s, and write in 1.76 s. This puts it between HDF5 and Parquet on read speed, and between Parquet and HDF5 on file size. DuckDB is a strong option for local analytical workflows where the full Snowflake stack is not available, but its embedded file format is not portable across different tools the way Parquet is.

Snowflake read times (0.42–0.60 s) are higher than the equivalent Python reads because they include query planning and distributed execution overhead. The relative ordering across formats is the same as in Python, with Parquet still the fastest. Snowflake write times are also higher than local Python writes because Snowflake encrypts and uploads data to its internal object store.

T5: Data augmentation

Each cleaned trip table was enriched with four categories of external contextual data: weather at pickup time, school density in the pickup zone, attraction density in the pickup zone, and the number of active permitted events in the pickup borough. All data was fetched directly inside Snowflake using Python stored procedures with external network access (`scripts/t5/t5_fetch_data.sql`), so no local ETL was required. The final enriched tables are stored as `GOLD.T5_YELLOW`, `GOLD.T5_GREEN`, `GOLD.T5_FHV`, and `GOLD.T5_FHVV`.

Taxi zone reference tables

Two reference tables are used in all spatial joins. `EXTERNAL_DATA.TAXI_ZONE_LOOKUP` is the standard TLC zone lookup CSV, downloaded directly from the TLC website and loaded into Snowflake as-is. It maps each of the 263 location IDs to a borough, zone name, and service zone. `EXTERNAL_DATA.TAXI_ZONES_GEOM` holds the actual polygon geometry for each zone, used for point-in-polygon lookups. The TLC shapefile was reprojected from EPSG:2263 to EPSG:4326 using GeoPandas (`scripts/t5/convert_taxi.ipynb`), serialised to WKT, and loaded into Snowflake where the geometry column was created with `TO_GEOGRAPHY()`.

External data sources

Daily weather data for NYC was fetched from the Open-Meteo historical API [4] in five 3-year chunks to stay within request size limits. Three variables were retrieved, daily maximum temperature, minimum temperature, and precipitation sum, and then joined to trips by pickup date.

School locations were fetched from the NYC Open Data portal (dataset `n3p6-zve2`, 2019–20 School Locations) via the Socrata API. Coordinates were extracted from the nested GeoJSON response and each school was spatially joined to its taxi zone using `ST_CONTAINS()`. The result is a school count per zone stored in `EXTERNAL_DATA.T5_SCHOOLS_BY_ZONE`.

Museum and attraction locations were fetched from NYC Open Data (dataset `fn6f-hvty`) using the same approach. The result is an attraction count per zone stored in `EXTERNAL_DATA.T5_ATTRACTIONS_BY_ZONE`.

Permitted public event data was fetched from NYC Open Data (dataset `bkfu-528j`) for 2020–2024 in quarterly chunks of 16 requests to respect the 10,000-row API limit [5]. Each event was expanded across its active hours in `scripts/t5/t5.sql` using the functions `ARRAY_GENERATE_RANGE` and `LATERAL FLATTEN`. The events with no recorded end time were assumed to last 3 hours. The

Storage format benchmark - Green taxi 2024 (617,885 rows)

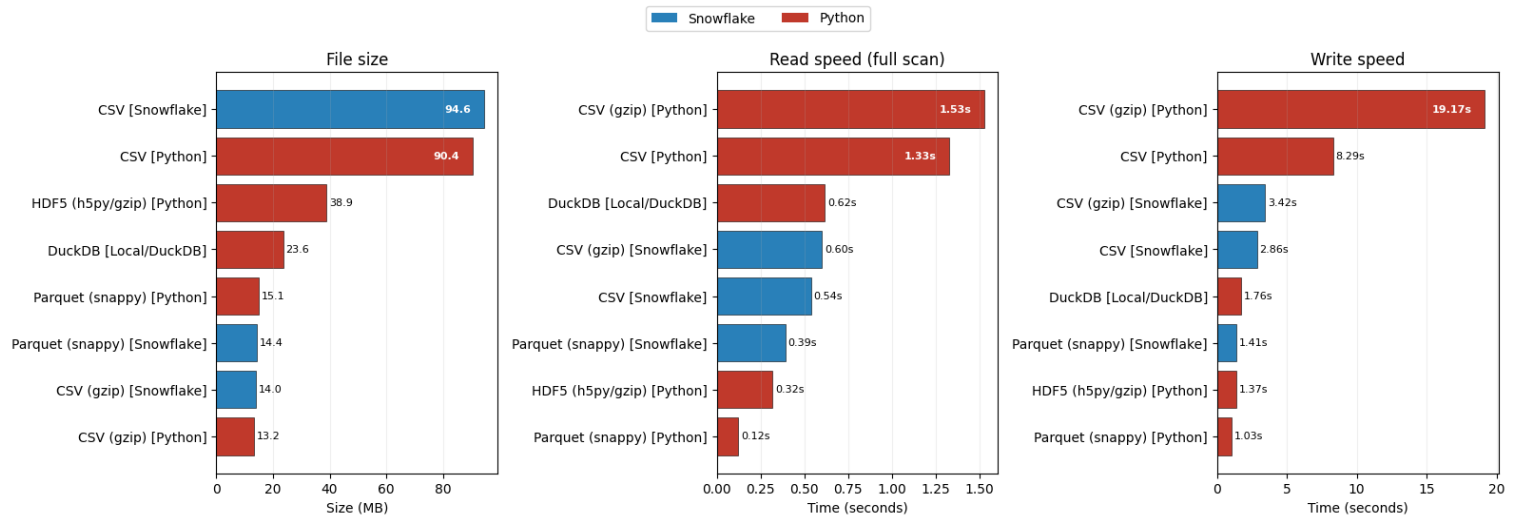


Figure 13. Format benchmark: file size, read speed, and write speed for all format/engine combinations.

result is an active event count per hour and borough, joined to trips by truncated pickup hour and borough.

Final enriched tables

The gold join script (`scripts/t5/t5_gold.sql`) temporarily scales the Snowflake warehouse to LARGE to handle base tables of up to 1.52B rows. Each output row contains all original trip columns plus BOROUGH, ZONE, SERVICE_ZONE, TEMP_MAX_C, TEMP_MIN_C, PRECIPITATION_MM, SCHOOL_COUNT, ATTRACTION_COUNT, and ACTIVE_EVENTS.

The next task was stream analysis, for which we aggregated events into hourly buckets and then computed rolling statistics over 24 hours, where each window included the current hour and the previous 23 hours. We calculated the count, mean, and standard deviation for the three key attributes: `trip_distance`, `trip_duration_sec`, and `fare_amount`. These calculations were performed separately for each borough and for the top 10 most active locations, which we selected by combining the number of pickups and dropoffs and choosing the most active ones. Figure 14 shows a preview of the rolling 24-hour statistics created in Snowflake.

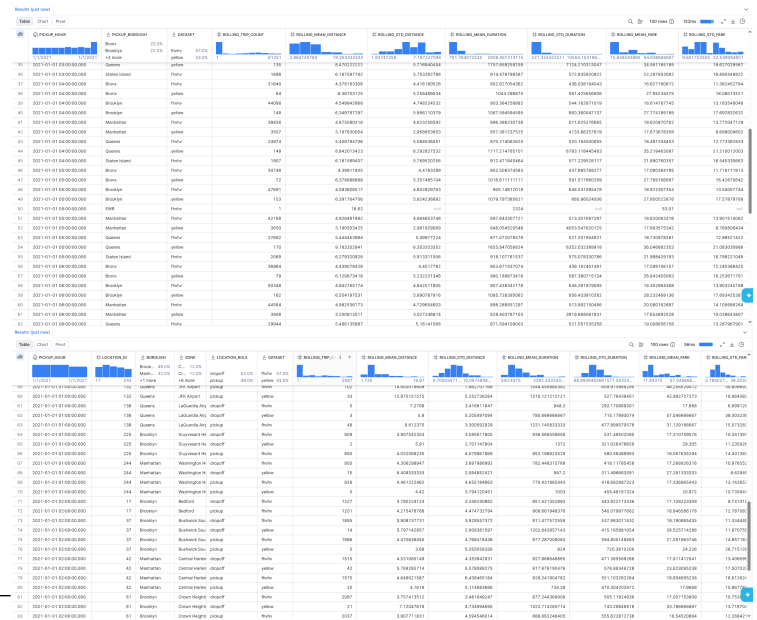
T6: Streaming with Kafka

We treated the historical trip information as a stream and chose the year 2021 to combine the two data sources: Yellow Taxi and the FHVHV dataset. Because using the streaming API and streaming into Snowflake adds additional costs, the producer was limited to a smaller sample, especially since we had to experiment with several approaches to make it work on Snowflake and did not want to increase costs by too much.

The streaming setup was implemented using Docker (configuration located in `scripts/t6/docker-compose.yml`) and Kafka, which included two Kafka brokers and a Kafka Connect service. Kafka Connect is used with the Snowflake Kafka Connector to ingest Kafka messages directly into Snowflake through the Snowpipe Streaming workflow.

We worked with two Kafka topics, `t6-yellow-2021` and `t6-fhvhv-2021`, so we could replay the records and monitor them independently. In Snowflake, we first created `GOLD.T6_STREAM_SOURCE_2021`, a table that combines records from `SILVER.YELLOW_TRIPS_CLEAN` and `SILVER.FHVVHV_TRIPS_CLEAN` into a common schema. We also added two extra columns: `dataset` and `stream_id`, which is based on pickup time order. The producer (`scripts/t6/producer_t6_2021.py`) reads this table, ordered by `stream_id`, to send each row as a JSON message to the correct Kafka topic in proper order, allowing us to simulate the streaming process.

To transfer data from Kafka to Snowflake, the Kafka Connector first writes messages to `GOLD.T6_KAFKA_LANDING_2021`, storing the JSON message in `RECORD_CONTENT` and key metadata (topic, partition, offset) in `RECORD_METADATA`. This table contains raw JSON, which is then parsed into standard columns in the view `GOLD.T6_STREAM_EVENTS_2021`.



values (dataset flag, cyclic hour and weekday encodings, pickup and dropoff location IDs, trip distance, trip duration, and fare amount).

During streaming, we trained the model with `n_clusters=None`, which allows it to build the internal tree structure incrementally. We then summarize the obtained results into final clusters, which we set to 6 in our case, so the results are quickly interpretable. Results are saved as CSV files in `scripts/t6` and also written back to Snowflake as tables. Figure 15 shows the final summarized results of the BIRCH stream clustering.

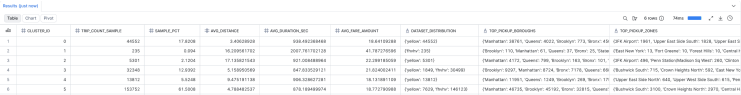


Figure 15. Summary of the final BIRCH stream clustering result.

From the figure, we can see that some clusters consist entirely of Yellow taxi trips and are oriented toward Manhattan or the airports, while others are dominated by FHVHV and are more dispersed. This indicates some overlap, but also distinct usage patterns.

T7: City-wide demand forecasting

Methodology

For the batch machine learning project, we chose the city-wide demand forecasting problem. Our goal was to predict hourly ride demand using information from all four datasets. This allows for better resource planning and helps us understand how demand changes depending on the type of service.

Using `scripts/t7/t7.sql`, we created our main feature table, `GOLD.T7_DEMAND_FEATURES`. Each entry represents one hourly demand observation for a service and a pickup zone: `dataset + pickup.location_id + pickup.hour`. The target was `TRIP_COUNT`, but for training, we transformed it to `LOG_TRIP_COUNT = LN(1 + TRIP_COUNT)` because demand is highly skewed, which means that many zone-hour combinations have low demand, while some have very high demand. For predictions, we scaled back to the original scale to have a fair evaluation.

Because we are working with temporal data, the data split for training and evaluation must be based on time, not random selection. For the training set, we used the common hourly range where all four datasets and the T5 weather data were available, starting in 2019, up to the beginning of 2024. For the validation set, we used data from 2024, and for the test set, data from 2025.

We also wanted to see how much the augmented feature set could influence and improve the final predictions. The baseline set included features such as service type, pickup zone, time features (weekend, rush-hour, night flags), cyclic encodings (hour, day, month cyclic features), lag features (1h, 24h, 168h), and rolling averages (24h, 168h). The augmented set included all of these, and also added external data from the T5 task, such as information about weather, number of schools, attractions, and active events.

For algorithm selection, we chose XGBoost as a native distributed third-party algorithm and `SGDRegressor.partial_fit` to represent scikit-learn algorithm with `partial_fit`. The third algorithm we included was PyTorch DDP (Distributed Data Parallel), implemented as a simple custom neural network in PyTorch. This was an alternative to the Dask-ML category, which we could not install or run in the Snowflake environment. Since PyTorch usage was documented under the distributed training on Snowflake, we included it for additional comparison. To perform the task in a distributed manner, XGBoost used Snowflake ML data connectors, PyTorch used sharded data connectors, and SGD read the data in batches.

We did our distributed experiments in a Snowflake Container Runtime using a Ray cluster, which is Snowflake’s method for scaling and utilizing distributed capabilities. It runs as a background process that we can easily connect to and scale dynamically during development. In our testing, we scaled the cluster to different numbers of workers and measured training time, prediction time, error metrics, process memory, and GPU memory.

Main model results and different experiments

Table 6 summarizes the most important runs. The main quality metrics used are MAE, RMSE, and R^2 , computed on the 2025 test set. Full results are available as CSV files in `scripts/t7_t10/`.

Table 6. Main T7 model results on the 2025 test set.

Model	Setup	Workers	Train s	MAE	RMSE	R^2	Proc. MB
XGBoost	global baseline GPU	4	134.69	9.40	29.96	0.852	1289.79
XGBoost	global augmented GPU	4	137.83	9.44	30.77	0.844	1304.43
XGBoost	separate augmented GPU	4	511.86	6.37	21.84	0.921	1422.24
PyTorch DDP	global baseline GPU	2	150.38	5.14	27.17	0.878	3241.93
PyTorch DDP	separate baseline GPU	2	183.39	8.51	69.82	0.194	2293.60
SGD	augmented partial_fit	1	69.68	64.62	417.44	-27.80	3261.07

Based on the results, the aggregated separate XGBoost models performed best. We trained one model for each service type and then aggregated their test errors, addressing our question of comparing a single global model to an aggregation of four separate models (FHV, FHVHV, Green, Yellow). For XGBoost, the separate approach gave better results: RMSE improved from 29.96 to 21.84, and R^2 increased from 0.852 to 0.921.

For PyTorch, the result was reversed: the single global PyTorch model performed better than the aggregated separate PyTorch models. This suggests that our neural network benefited from a combined training set and shared patterns. For the architecture, we used a simple feed-forward neural network with two hidden layers of size 64, ReLU activations, and one output neuron. For training, we used the Adam optimizer and mean squared error loss.

Table 7 presents the results in a separate table. We display the RMSE metric because it penalizes large forecasting errors and outliers, which are particularly problematic when planning resources in our case.

Table 7. Single global model vs. aggregation of four separate models.

Algorithm	Global RMSE	Separate RMSE	Better approach
XGBoost	29.96	21.84	Separate models
PyTorch DDP	27.17	69.82	Global model

For augmentation, we found that it did not help the XGBoost model, as the RMSE increased from 29.96 to 30.77. This suggests that the most important information for modeling is already captured by recent demand and weekly patterns, so additional augmentation does not provide any useful benefit for this model. For PyTorch (4-worker run), augmentation was helpful, as the RMSE decreased from 68.04 to 44.52, and for SGD it decreased from 563.01 to 417.44, so the external features were useful for these two models.

To test scalability, we increased the number of Ray cluster workers. For XGBoost, we could not run it with only one worker due to insufficient memory, so we tested with two and four GPU workers. This reduced training time, and accuracy remained about the same. For PyTorch, training time decreased even further. Increasing from one to two workers also improved model accuracy, which was another significant benefit. Full results are shown in Table 8.

SGD was used as a simple baseline because it is not natively distributed. The model is sequential, and with Ray on Snowflake, we cannot automatically synchronize a shared scikit-learn model across all workers. Because of that, we tested SGD with only one worker and

Table 8. Worker-scaling results for GPU models.

Algorithm	Setup	Workers	Train s	RMSE
XGBoost	global baseline	2	146.80	29.92
XGBoost	global baseline	4	134.69	29.96
PyTorch DDP	global baseline	1	201.07	109.94
PyTorch DDP	global baseline	2	150.38	27.17
PyTorch DDP	global baseline	4	116.61	68.04

focused on the other two models (XGBoost and PyTorch) for more advanced evaluations and testing under different conditions. As expected, SGD is quite fast, but it is not accurate enough, as it struggles significantly with the nonlinear problem we are addressing.

Overall, we solved this problem using machine learning approaches and demonstrated that we can estimate expected demand across the entire city and test different modeling strategies, which depend on the model, to determine which performs best.

T8: Impact of FHVHV on traditional services

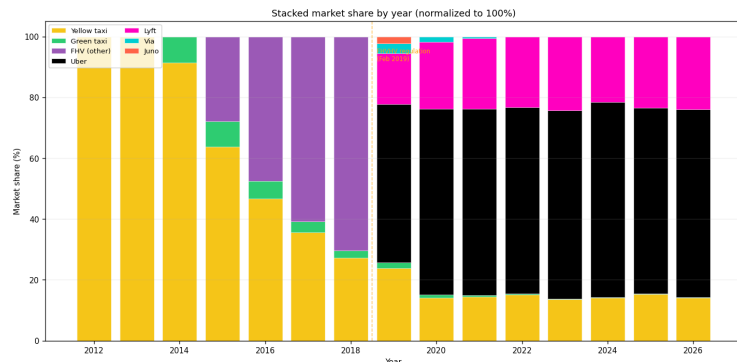
Regulatory context

Before February 2019, Uber and Lyft reported under the FHV dataset. The FHVHV regulation required operators dispatching more than 10,000 trips per day to obtain a new HVFHS licence and report separately. FHV monthly counts immediately dropped from around 24M to 1–2M, while FHVHV simultaneously emerged at 18–20M trips per month. Any analysis must treat this as a structural break, not a real volume change. Two smaller operators, Juno (HV0002) and Via (HV0004), briefly appeared in FHVHV data with under 2% share before exiting NYC in 2019.

Market share evolution

Yellow taxi peaked at around 170M trips per year in 2012–2013 with effectively 100% market share. By 2018 it had fallen to around 82M (–52%) and by 2023–2024 to around 40M, a 76% drop from peak. Green never exceeded 20M trips per year and has since fallen to near zero. After the 2019 transition, Uber settled at 160–180M trips per year and Lyft at 50–65M.

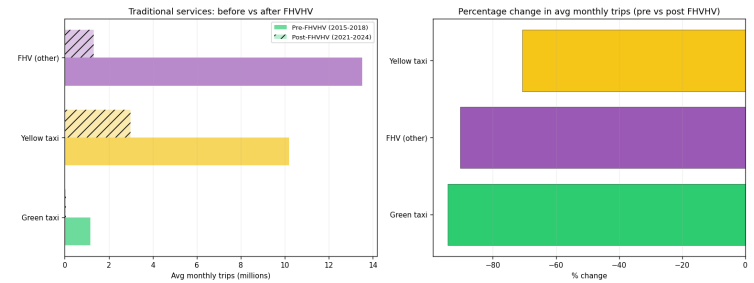
Figure 16 shows the stacked annual share. Yellow held close to 100% in 2012, fell below 50% by mid-2016, and was at around 22% just before the FHVHV transition. Uber immediately captured around 50% of all tracked trips in 2019 and has since stabilised at 55–60%. Lyft holds a steady 20–22%, Yellow around 13–15%, and Green below 1%.

**Figure 16.** Stacked annual market share by operator.

Before vs. after FHVHV

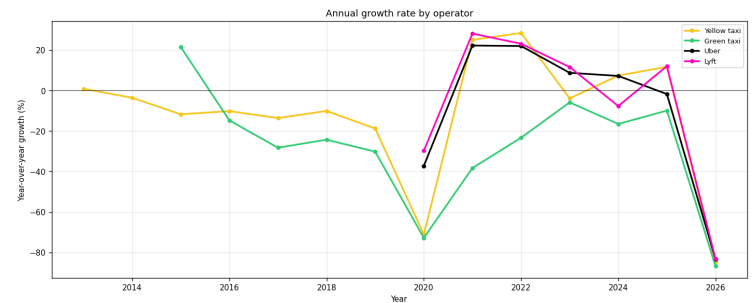
To isolate the FHVHV impact from COVID distortions, we compared average monthly trips in 2015–2018 against 2021–2024. Figure 17

shows the result. Yellow fell from around 10.5M to 3.0M monthly trips (–71%). Green fell from 1.0M to near zero (–95%). FHV (other) fell –88%, though most of this reflects the regulatory reclassification of Uber and Lyft rather than actual volume loss.

**Figure 17.** Avg. monthly trips: pre-FHVHV (2015–2018) vs. post-FHVHV (2021–2024).

Annual growth rates

Figure 18 shows year-over-year growth rates. Yellow declined around –10% per year from 2013 to 2018, crashed –72% in 2020, and recovered +23% in 2021. Green dropped –15 to –30% per year throughout, with a –75% COVID crash. Uber and Lyft contracted –40% in 2020, rebounded +20–25% in 2021, and have been broadly stable since.

**Figure 18.** Year-over-year trip growth rates for Yellow, Green, Uber, and Lyft.

T9: Geographic visualization

T9 maps the T2 quality results and selected T3 findings onto NYC's 263 taxi zones. Maps were produced in `scripts/t9/t9_map.ipynb` using `matplotlib`; zone geometries were fetched via `ST_ASGEOMETRY()` from `EXTERNAL_DATA.TAXI_ZONES_GEOM` and rendered as polygon patches. Supporting aggregation tables were created in `scripts/t9/t9.sql`.

T2: Data quality per zone

The T2 quality checks were re-applied per pickup zone against the raw `SILVER` tables and stored in `GOLD.T9_QUALITY_BY_ZONE`. Four issue types were summed per zone; zero distance, pickup equals dropoff, dropoff before pickup, and negative fare, into a single percentage score. The colour scale is clamped at the 95th percentile so outlier zones do not compress the rest of the palette.

Figure 19 shows the result. FHVHV is uniformly clean across the city. Yellow has a few isolated hotspots near Midtown and the airports. FHV has the most widespread elevated rates, particularly in zones with historically high rates of missing location data.

T3: Trip volume per zone

Trip volume per zone across all years is taken from `GOLD.T3_PICKUP_LOCATION_DIST` and plotted on a logarithmic colour scale (Blues),

T2 data quality issues by taxi zone

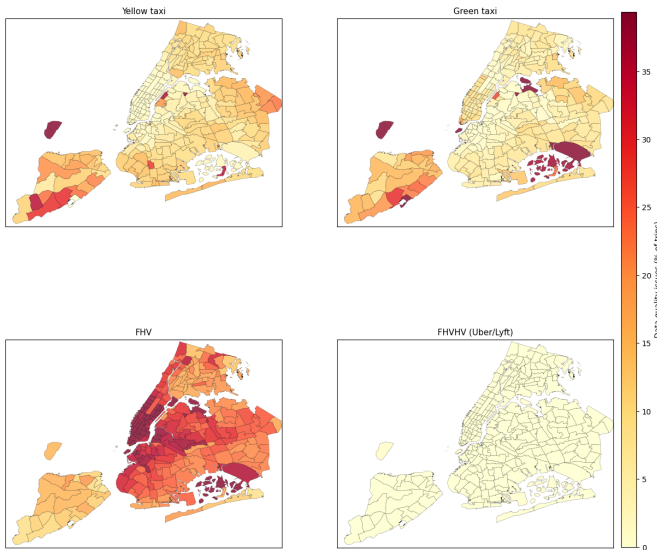


Figure 19. T2 quality issue rate per zone, all four datasets.

since the range between Manhattan and outer-borough zones spans several orders of magnitude.

Trip volume by taxi zone (from T3)

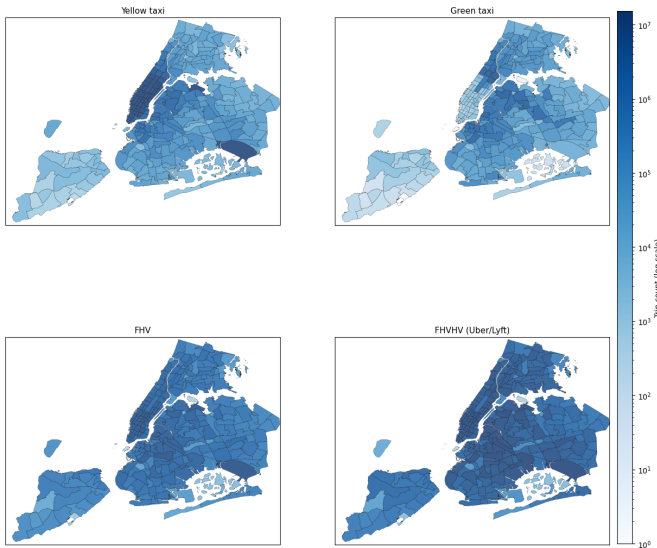


Figure 20. T3 trip volume per zone, all years, log scale.

Figure 20 confirms the T3 spatial findings: Yellow concentrates in Manhattan and airports, Green in the outer boroughs, FHV is diffuse across all boroughs, and FHVHV has the most uniform citywide coverage.

T3: COVID-19 impact per zone

Figure 21 shows the percentage change in Yellow taxi trips per zone between 2019 and 2020, from `GOLD.T9_COVID_BY_ZONE`. The diverging RdYlGn scale is centred at -70% since the decline was severe everywhere; green zones declined less, red zones more.

Airport and Midtown zones, which depend on business and tourist travel, saw the steepest declines. Residential outer-borough zones show relatively smaller drops, partly because their pre-COVID baseline was already low.

COVID-19 impact: yellow taxi trip change 2020 vs 2019 (%)
(green = less decline, red = severe decline)

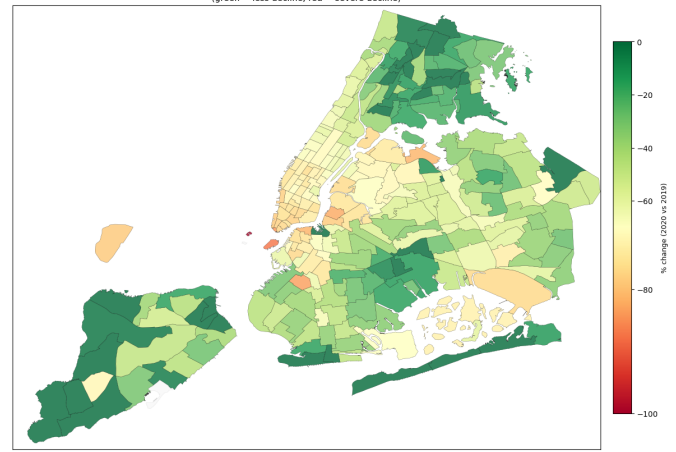


Figure 21. Yellow taxi trip change 2020 vs. 2019 by zone.

T3: Peak hour per zone

Figure 22 shows the peak Yellow taxi pickup hour per zone, from `GOLD.T9_HOURLY_BY_ZONE`. The twilight-shifted colormap places midnight at both ends and noon at the centre.

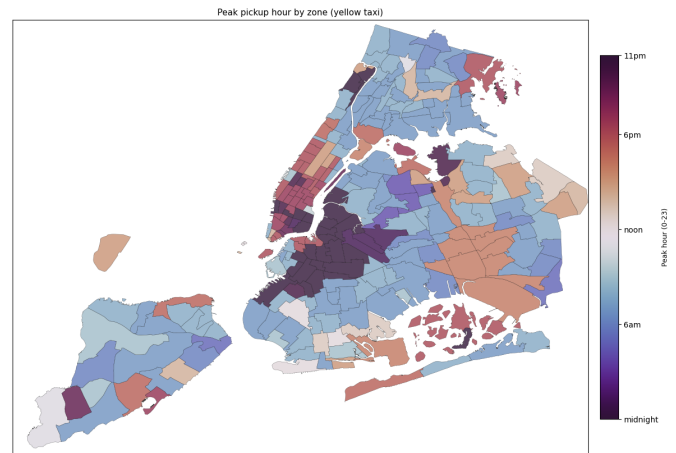


Figure 22. Peak Yellow taxi pickup hour by zone (0–23).

Most Manhattan zones peak in the evening (18:00–20:00), consistent with T3. Airport zones peak in the afternoon due to flight schedules. Business district zones show a morning peak around 08:00–09:00.

T3: Post-COVID recovery per zone

Post-COVID recovery: 2024 avg monthly trips as % of 2019
(green = fully recovered, red = still below pre-COVID)

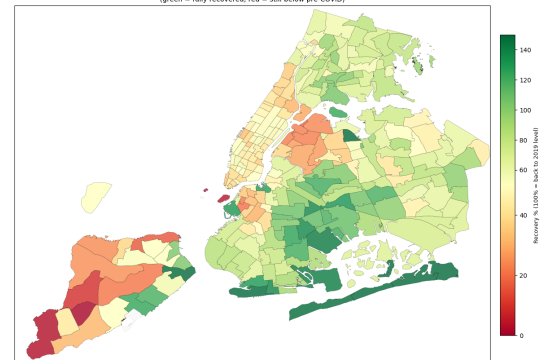


Figure 23. Yellow taxi recovery: 2024 avg. monthly trips as % of 2019.

Figure 23 shows 2024 average monthly Yellow taxi trips as a percentage of 2019, from `GOLD.T9_RECOVERY_BY_ZONE`. The scale

is centred at 50% to reflect that citywide Yellow recovery is well below full. Values above 100% mean the zone has exceeded its 2019 level.

Most zones remain below 50% recovery. Tourist-adjacent zones show stronger recovery; Midtown and Financial District zones remain furthest below their 2019 baselines.

T10: Distributed CPU and GPU processing

This is an extension of the T7 forecasting task, except that we now focus on distributed CPU/GPU execution and scalability, while the rest of the workflow remains the same as previously described. The main component enabling this is the Snowflake Container Runtime using Ray clusters, which we used for distributed XGBoost GPU training, distributed XGBoost CPU training, and distributed PyTorch DDP GPU training. Table 9 shows key results. The memory columns report the peak process memory and peak GPU memory observed during the run.

Table 9. Distributed CPU/GPU and scalability results for T10.

Model	Compute setup	Workers	Train s	RMSE	R^2	Proc. MB	GPU MB
XGBoost	global augmented CPU	4	154.43	33.82	0.811	594.00	0
XGBoost	global augmented GPU	4	137.83	30.77	0.844	1304.43	528
XGBoost	global baseline GPU	2	146.80	29.92	0.852	1219.78	769
XGBoost	global baseline GPU	4	134.69	29.96	0.852	1289.79	514
PyTorch DDP	global baseline GPU	1	201.07	109.94	-0.996	3743.36	24.93
PyTorch DDP	global baseline GPU	2	150.38	27.17	0.878	3241.93	24.93
PyTorch DDP	global baseline GPU	4	116.61	68.04	0.235	2791.51	24.93

The CPU/GPU comparison shows that GPU-based distributed XGBoost performed better in this setup: training time decreased from 154.43 s to 137.83 s, and RMSE improved from 33.82 to 30.77. Increasing the number of GPU workers was also beneficial. As previously observed in T7, PyTorch DDP achieved greater speedup in training time and improved accuracy when increasing the number of workers from 1 to 2. This demonstrates that the T7 forecasting task was executed with distributed CPU and GPU processing, with distributed XGBoost proving to be the most reliable model.

Conclusion

We built a complete big data pipeline for approximately 3.7 billion NYC TLC trip records, including ingestion, cleaning, exploration, augmentation, streaming, machine learning, and visualization. The main data finding is that NYC ride demand changed in two significant ways. The February 2019 FHVHV reporting change separated Uber, Lyft, and similar services from the FHV category, making app-based traffic visible as its own market. The second major change was COVID-19, which caused a significant loss of volume, especially for Yellow and Green taxis, which never fully recovered. We observed that demand still follows almost the same daily and weekly rhythms, but spatial patterns are very different, as also seen during BIRCH stream clustering. Yellow taxis are mainly concentrated in Manhattan and airport-related areas, Green taxis operate outside Manhattan's core, and FHV/FHVHV services have much broader citywide coverage. Technical results showed that Parquet with Snappy compression was the best storage format for our workload, and distributed XGBoost was the best forecasting model. We also found that the best XGBoost results came from training separate models for each dataset. External augmentation provided richer features, but did not improve every machine learning model.

Experience with Snowflake

We think that using Snowflake was a good choice, and overall we had a very positive learning experience with it. We completed the regular homework assignments, which included almost the same tasks, entirely on the Arnes HPC cluster, and found that Snowflake was generally much more intuitive and interactive. On the HPC cluster, you work directly with large Parquet files, which usually requires preparing scripts,

submitting jobs, waiting for results, and then separately checking the results. In Snowflake, you interact directly with tables; the data is always available, and you can easily inspect and perform many visual checks, which are almost immediate in the web interface.

It also works very well as a big data platform because you have elastic warehouses that you can scale up for expensive joins and scale down for regular queries. There are also many useful built-in functions (for example, for geospatial analysis in T5 and T9) and SQL features that help replace much of the custom Python logic. For the entire execution of the project, we used 32.3 compute credits, and our total Snowflake storage was 203.9 GB.

There are still some difficulties, and the main one is that Snowflake is a complex ecosystem with its own learning curve. You immediately notice that it uses its own version of SQL, which is close to standard SQL, but some operations must be done in a Snowflake-specific way, so you need to learn those patterns. This is helpful in the long run, but it takes time to discover everything and learn how to use features effectively so they actually benefit you. The documentation can sometimes be overwhelming and cause information overload, either because there are not enough examples or because there are too many different ways to solve a similar problem. The hardest part was the machine learning, as it required extensive documentation review to set up distributed training of ML models and properly use the Ray cluster, so the T7 and T10 tasks required the most experimentation and time to solve. Other limitations included that Snowflake does not directly support every format or workflow we wanted to benchmark (such as HDF5 and DuckDB), so we performed those tasks in Python. Streaming also introduces additional complexity and costs, so we worked with a limited streamed sample during development to avoid unnecessary expenses.

Overall, we are very satisfied with our decision to work with Snowflake and appreciate the opportunity to learn more about this extremely powerful platform. It made it much easier to combine large-scale SQL processing, data organization, visualizations, external data integration, and machine learning within a single unified environment. While it does take more time to learn initially, and as tasks become more complex its disadvantages start to appear, for simpler tasks and most analytical work it is a highly beneficial platform that accelerates workflow. There are also many features we did not need for this project, so there is still much we do not know and could use for other types of projects.

References

- [1] New York City Taxi and Limousine Commission. *TLC Trip Record Data*. <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>. Accessed: 2026.
- [2] Snowflake Inc. *Snowflake: The AI Data Cloud*. <https://www.snowflake.com>. Accessed: 2026.
- [3] Apache Software Foundation. *Apache Arrow*. <https://arrow.apache.org>. Accessed: 2026.
- [4] Open-Meteo. *Open-Meteo Historical Weather API*. <https://open-meteo.com>. Accessed: 2026.
- [5] City of New York. *NYC Open Data*. <https://opendata.cityofnewyork.us>. Accessed: 2026.